

VSMS Connect — HTTP Connector API

Reference for the generic HTTP connector — the
`/api/v1/businesses/:businessId/fiscalise` surface an external POS integrator talks to.

Generated 2026-07-05

Getting started

The HTTP connector is a machine-to-machine API. Before your integration code can call it, a human admin at the customer's company must complete a one-time setup in our app to obtain three credentials.

Step 1 — Account + business onboarding (one-time, per customer)

The admin signs up at our app, verifies their email, and registers their business (uploads V-SDC certificate, validates PAC, enters TaxCore UID + licence key). This produces a unique BUSINESS_ID UUID tied to the business's fiscalisation credentials.

Step 2 — Generate an HTTP-scoped API key

In the app: **Integrations** → **Generic HTTP** → **Issue new key**. The admin picks a **Location** from the dropdown — the key is bound to that location's default V-SDC certificate and fiscal counter (one key per shop / outlet). A business with multiple locations issues one key per location. A one-shot reveal modal then displays three values, each with a copy button:

- **Backend URL** — e.g. `https://api.vsmsconnect.com/api/v1`
- **Business ID** — UUID, e.g. `7f3a1e2c-...`
- **API key** — 64-char hex string, e.g.
`a3f2b1c8d9e7f6a4b2c1d8e9f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8`

The API key is shown **once**. The customer's admin must copy it and hand it (plus the Backend URL and Business ID) to their integration team — usually via their internal password manager. If the key is lost, the admin revokes it and issues a new one. Backend URL and Business ID remain visible in the app's API Keys list.

Step 3 — Configure your integration

Set three environment variables (or config equivalents) in your code:

```
VSMS_CONNECT_BACKEND_URL = "https://api.vsmsconnect.com/api/v1"
VSMS_CONNECT_BUSINESS_ID = "7f3a1e2c-..."
VSMS_CONNECT_API_KEY     = "a3f2b1c8d9e7f6a4..."
```

Every request goes to:

```
POST {VSMS_CONNECT_BACKEND_URL}/businesses/{VSMS_CONNECT_BUSINESS_ID}/fiscalise
```

with headers:

```
Authorization: ApiKey {VSMS_CONNECT_API_KEY}
Idempotency-Key: <a uuid you generate per request>
Content-Type: application/json
```

Per-location keys

API keys are **location-scoped**. A business with multiple shops registers each shop as a `Location` in the app and issues a separate API key per location — the key binds every fiscalisation request to that location's default V-SDC certificate, fiscal counter, and receipt header. Multi-location customers therefore end up with one `Business ID` + `N` API keys (one per location). Your integration code typically routes the right API key per outlet via configuration; the Backend URL and `Business ID` are constant across the customer's whole estate.

Per-request location override. Alternatively, a business-level key (one issued without a location) may name the target location on each request via the optional `locationId` body field on `POST /fiscalise`. When present it selects that location's default certificate (the id must belong to the business, else `422 LOCATION_NOT_FOUND`) and wins over the key's own location — so one cloud integration can fiscalise for many outlets with a single credential. With a **location-scoped key** you don't need this field at all — every sale on that key already runs under its location; `locationId` is purely the override for a business-level key. Either way it only affects a fresh **sale**: refunds and copies resolve their location from neither the key nor the field — they inherit the original sale's location and certificate automatically.

Multi-tenant integrations

If your software integrates with multiple of our customers, each customer's admin completes Step 1 + Step 2 independently. You end up with one `Business ID` per customer plus one API key per (customer, location) pair. The Backend URL is the same for every customer.

Tax labels

Each line item carries a `taxLabel` — the V-SDC label, sent directly. No admin tax-mapping configuration is required for this connector (matches CSV import semantics). The valid label set:

label	meaning
A	Standard rate (Vanuatu VAT 15%)
B	Reserved (rate > 0, label B)
C	Reserved (rate > 0, label C)

label	meaning
D	Reserved (rate > 0, label D)
E	Reserved (rate > 0, label E)
F	Reserved (rate > 0, label F)
N	Zero-rated (rate = 0)
P	Exempt
T	Out of scope

Case matrix

One endpoint, `POST /fiscalise`, handles every fiscalisation case via two body fields: `invoiceType` (NORMAL / COPY / PROFORMA / ADVANCE) and `transactionType` (SALE / REFUND), plus an optional `training: true` flag that overrides `invoiceType` to TRAINING. The other three routes are operational helpers:

- `GET /fiscalise/:invoiceId` — idempotent status retrieval after a 202 from POST.
- `POST /fiscalise/:invoiceId/trigger` — explicit dispatch for a PROFORMA imported via the 201 path.
- `POST /fiscalise/cancel` — submit a V-SDC cancellation document by `fiscalInvoiceNumber`.

POST**/api/v1/businesses/:businessId/fiscalise** **auth:** apikey (scope=http)

Main ingestion endpoint. Persists the invoice + dispatches it to V-SDC and sync-waits up to ~10 s for the fiscal response. Returns 200 with `fiscalInvoiceNumber` + QR/URL/hash when the consumer reaches a terminal state inside the window; 202 with a `statusUrl` when the timeout elapses with payments still in-flight; 201 with a `triggerUrl` for PROFORMA bodies (caller must follow up with the trigger endpoint).

handlers: FiscaliseController.fiscalise

INPUTS

name	required	default	description
businessId path uuid	yes	—	Target business UUID (must match the API key's BusinessId).
Authorization header string	yes	—	`ApiKey <raw key>` — key must be scoped to `ConnectorType.Http`.
Idempotency-Key header string	no	—	Stable UUID per request. A replay within ~5 min returns the cached response without re-running the handler.
sync_timeout_ms query number	no	10000	Sync-wait window in milliseconds. Capped at 30000.
invoiceNumber body string	yes	—	Caller's invoice number. Printed on the V-SDC receipt AND used as the dedup key — `SHA-256(businessId:http:invoiceNumber)` becomes the persisted `IdempotencyKey`. Re-sending the same value returns `409 INVOICE_DUPLICATE`.
invoiceType body 'NORMAL' 'COPY' 'PROFORMA' 'ADVANCE'	yes	—	Case selection. TRAINING is a separate top-level flag — not an invoiceType value.
transactionType body 'SALE' 'REFUND'	yes	—	Sale or refund.
training body boolean	no	—	When `true`, stamps `invoiceType = TRAINING` regardless of the caller-supplied value. Mutually exclusive with `invoiceType: 'COPY'` (422 `TRAINING_COPY_NOT_SUPPORTED`).
source body { referencedFiscalNumber: string; referencedFiscalTimestamp: string }	no	—	Required when `invoiceType=COPY`; forbidden otherwise. Carries the source payment's V-SDC fiscal number +

name	required	default	description
			timestamp (stored from a prior 200 response).
referentDocumentNumber body string	no	—	Required when `transactionType=REFUND` and the source was externally-fiscalised (paired with `referentDocumentDT`).
referentDocumentDT body string	no	—	ISO-8601 timestamp of the referent document. Pairs with `referentDocumentNumber`.
reference body string	no	—	PROFORMA → NORMAL conversion key. The fiscalisation module's `linkInvoiceToQuote` stamps `ConvertedFromQuoteld` and carries the proforma's SDC referent through automatically.
locationId body uuid	no	—	Optional per-request location selector (multi-location). On a fresh SALE it picks which location's default certificate signs the invoice; the value must belong to the business (else 422 LOCATION_NOT_FOUND) and wins over the API key's location. Omit to use the API key's location, or the business default. Forbidden on COPY (a copy inherits its source's location) and ignored on REFUND / advance-append.
invoiceDate body number string	yes	—	Epoch ms or ISO-8601. Required for non-COPY.
currencyCode body string	yes	—	ISO-4217 three-letter currency code. Required for non-COPY.
cashierId body string	yes	—	Cashier / operator id sent to V-SDC as the `cashier` field — printed on every fiscal receipt as the 'Cashier:' line. Required everywhere (Vanuatu V-SDC mandate); whitespace-only values are rejected with `VALIDATION_ERROR`.
buyer body { tin?: string; name?: string; costCentrelId?: string; email?: string } null	no	—	Optional. Presence triggers the 'with buyer' V-SDC wire body. V-SDC carries TIN + name + costCentrelId on the wire receipt; `email` is stored locally and used by the auto-email pipeline to send the customer a copy of the fiscalised receipt when the per-connector auto-email toggle is enabled.

name	required	default	description
lineItems body Array<{ description, quantity, unitPrice, taxLabel, taxRatePercent, lineSubtotal, lineTaxAmount, lineTotal, gtin?, sortOrder?, itemAdditionalFields? }>	yes	—	Required non-empty for non-COPY. `taxLabel` must be one of the V-SDC labels (A/B/C/D/E/F/N/P/T).
payments body Array<{ amount, paymentType, paymentDate, externalPaymentId?, tenders? }>	yes	—	Required non-empty for non-COPY. For ADVANCE deposit chains: send ONE invoice with multiple payments — the consumer auto-chains ADVANCE → ADVANCE → ... → NORMAL. Each payment may carry a `tenders[]` breakdown for split-tender.
subtotalAmount body number	yes	—	Sum of `lineItems[].lineSubtotal` (±0.01 reconciliation).
taxAmount body number	yes	—	Sum of `lineItems[].lineTaxAmount` (±0.01 reconciliation).
totalAmount body number	yes	—	Sum of `lineItems[].lineTotal` (±0.01 reconciliation). Must also equal sum of `payments[].amount`.
invoiceAdditionalFields body Record<string, string>	no	—	Optional bag forwarded verbatim to V-SDC as `invoiceAdditionalFields`.
paymentAdditionalFields body Record<string, string>	no	—	Optional bag forwarded verbatim to V-SDC inside the payment array.

OUTPUTS

200

APIResponseDataObject<FiscaliseSuccessResponse>

Every payment reached a terminal state inside the sync-wait window. `paymentResults[].fiscalInvoiceNumber` carries the SDC reference — store it for later refunds / copies / cancellations.

201

APIResponseDataObject<FiscaliseAcceptedResponse>

PROFORMA imported but not auto-dispatched. Body includes `triggerUrl`; call it via `POST /fiscalise/:invoiceId/trigger` to fiscalise.

202

APIResponseDataObject<FiscaliseSuccessResponse>

Timeout elapsed with payments still in-flight. Body includes `statusUrl`; poll `GET /fiscalise/:invoiceId` for the terminal state.

409

APIError

`INVOICE\_DUPLICATE` — an invoice with this `invoiceNumber` already exists for this business.

422

APIError

`VALIDATION\_ERROR` — schema or cross-field validation failed. Inspect `validationErrors[]` for per-field codes ('REFUND_REFERENT_REQUIRED', 'INVALID_COPY_BODY', 'TRAINING_COPY_NOT_SUPPORTED', 'LINE_SUM_MISMATCH', 'PAYMENT_SUM_MISMATCH', 'TENDER_SUM_MISMATCH', 'LOCATION_NOT_FOUND' — the supplied `locationId` does not belong to this business).

502

APIError

`FISCAL\_ERROR` — V-SDC rejected every payment. Inspect `GET /fiscalise/:invoiceId` for per-payment error detail.

GET `/api/v1/businesses/:businessId/fiscalise/:invoiceId` **auth:** apikey (scope=http)

Idempotent status retrieval. Returns the same response shape as `POST /fiscalise` so a caller polling after a 202 gets a drop-in replacement once the consumer reaches a terminal state.

handlers: FiscaliseController.getStatus

INPUTS

name	required	default	description
businessId path uuid	yes	—	Target business UUID.
invoiceId path uuid	yes	—	Invoice id returned by a prior POST /fiscalise.
Authorization header string	yes	—	`ApiKey <raw key>` scoped to `ConnectorType.Http`.

OUTPUTS

200

APIResponseDataObject<FiscaliseSuccessResponse>

Current per-payment snapshot.

404

APIError

`INVOICE\_NOT\_FOUND` — invoice id is unknown under this business.

POST**/api/v1/businesses/:businessId/fiscalise/:invoiceId/trigger**

auth: apikey (scope=http)

Explicit dispatch for a PROFORMA imported via POST /fiscalise's 201 path. Delegates to the existing `triggerFiscalisationByInvoiceIds` primitive — non-PROFORMA invoices are rejected with 422 INVOICE_NOT_QUOTE.

handlers: FiscaliseController.trigger

INPUTS

name	required	default	description
businessId path uuid	yes	—	Target business UUID.
invoiceId path uuid	yes	—	PROFORMA invoice id.
Authorization header string	yes	—	`ApiKey <raw key>` scoped to `ConnectorType.Http`.
Idempotency-Key header string	no	—	Stable UUID per request.

OUTPUTS**202****APIResponseDataObject**<{ invoiceId; outcome; jobId; statusUrl }>

Dispatched. Poll `statusUrl` for terminal state.

422**APIError**

`INVOICE\_NOT\_QUOTE` — only PROFORMA invoices are eligible for this endpoint.

POST**/api/v1/businesses/:businessId/fiscalise/cancel** auth: **apikey** (scope=http)

Submit a V-SDC cancellation document for a previously-fiscalised payment. Resolves ``fiscalInvoiceNumber`` via ``findByFiscalInvoiceNumber``, delegates to the per-payment ``cancelPayment`` primitive (sibling `InvoicePayments` row with ``CancelsPaymentId``), and waits for the cancellation receipt to be signed before responding — so the caller gets back the full ``fiscalJournal`` + QR + signed hash in one round-trip and can print the cancellation receipt without polling. Falls through to 202 + ``statusUrl`` if the sync-wait window elapses.

handlers: `FiscaliseController.cancel`

INPUTS

name	required	default	description
<code>businessId</code> path uuid	yes	—	Target business UUID.
<code>Authorization</code> header string	yes	—	<code>`ApiKey <raw key>`</code> scoped to <code>`ConnectorType.Http`</code> .
<code>Idempotency-Key</code> header string	no	—	Stable UUID per request.
<code>fiscalInvoiceNumber</code> body string	yes	—	SDC <code>`fiscalInvoiceNumber`</code> returned by a prior 200 response.

OUTPUTS

200

`APIResponseDataObject<FiscaliseCancelResponse>`

Cancellation reached terminal state inside the sync-wait window. ``paymentResults[0]`` carries the cancellation receipt's own SDC ``fiscalInvoiceNumber``, ``fiscalJournal`` (textual receipt), ``fiscalQrCode``, ``fiscalVerificationUrl``, ``fiscalSignedHash``, and ``fiscalTimestamp``. ``cancellationPaymentId`` echoes the new sibling payment row for correlation.

202

`APIResponseDataObject<FiscaliseCancelResponse>`

Timeout elapsed with the cancellation row still in-flight. ``statusUrl`` (``GET /fiscalise/:invoiceId``) returns the terminal cancellation once signed.

404

`APIError`

``INVOICE_NOT_FOUND`` — no payment with that fiscal number exists under this business.

409

`APIError`

``INVOICE_CANCELLATION_IN_FLIGHT`` — another cancellation for the same payment is already in flight.

422

`APIError`

`INVOICE\_NOT\_CANCELLABLE` — payment is not fiscalised yet, missing fiscal reference, or wrong invoiceType. Body carries the specific code.